

# Library Management System

## Core Java & File I/O Implementation

A comprehensive project report detailing the design, architecture, and implementation of a dependency-free digital library system.

---

SUBMITTED BY

**Aarushi Jain**

ROLL NUMBER

**25BAI11558**

# 2. Introduction

---

## Project Overview

---

The Library Management System is a lightweight, console-based application built purely in Java. It allows library administrators to manage records without the overhead of external database servers.

## Technology Stack

---

- **Language:** Core Java (JDK 8+)
- **Data Storage:** Flat-file architecture (.txt files)
- **Libraries:** `java.io.*` and `java.util.Scanner`
- **Dependencies:** None (Zero external setup required)

# 3. Problem Statement

---

## ⚠ The Legacy Challenge

Traditional small-scale libraries often rely on manual ledgers. This approach is highly error-prone, making it difficult to track book availability, manage member data, and prevent record loss.

Conversely, setting up enterprise SQL databases (like MySQL/Oracle) is often overkill, expensive, and requires significant technical configuration for simple utility projects.

## 💡 The Proposed Solution

A digital transition bridging the gap. This project introduces a **lightweight, file-based system** that provides the structural benefits of a database (CRUD operations) while maintaining the absolute portability and zero-configuration nature of plain text files.

## 4. Functional Requirements

---

-  **Create Records:** The system must allow administrators to add new books (ID, Title, Author) and new members (ID, Name) via interactive console prompts.
-  **Read/View Records:** Users must be able to view a complete, formatted list of all registered books and active members retrieved from the local disk.
-  **Update Data:** The application must support inline editing, allowing users to search by ID and update specific book details while retaining old data if input is left blank.
-  **Delete Records:** Administrators must be able to permanently remove a book or member from the system using their unique identifier.
-  **Persistent Storage:** All changes must survive application restarts by appending/modifying data directly in `books.txt` and `members.txt`.

# 5. Non-functional Requirements

---



## Performance

The system must process read/write operations swiftly. Utilizing `BufferedReader` and `PrintWriter` ensures that parsing text lines occurs in milliseconds without high memory consumption.



## Portability

Designed to be OS-agnostic. By strictly relying on the standard JVM and avoiding external drivers, the application runs identically on Windows, macOS, or Linux.



## Reliability

Data integrity is paramount. The system must automatically generate required storage files upon startup (`initFiles()`) if they are missing, preventing fatal crash errors.

## 6. System Architecture

---

### Client & Controller Layer

The **Client Interface** is completely console-based. It utilizes `java.util.Scanner` to capture standard input and display continuous interactive menus in the terminal.

The **Controller (LibrarySystem)** contains the main application ``while`` loop. It routes user choices to specific logical methods (e.g., `addBook()`, `updateBook()`).

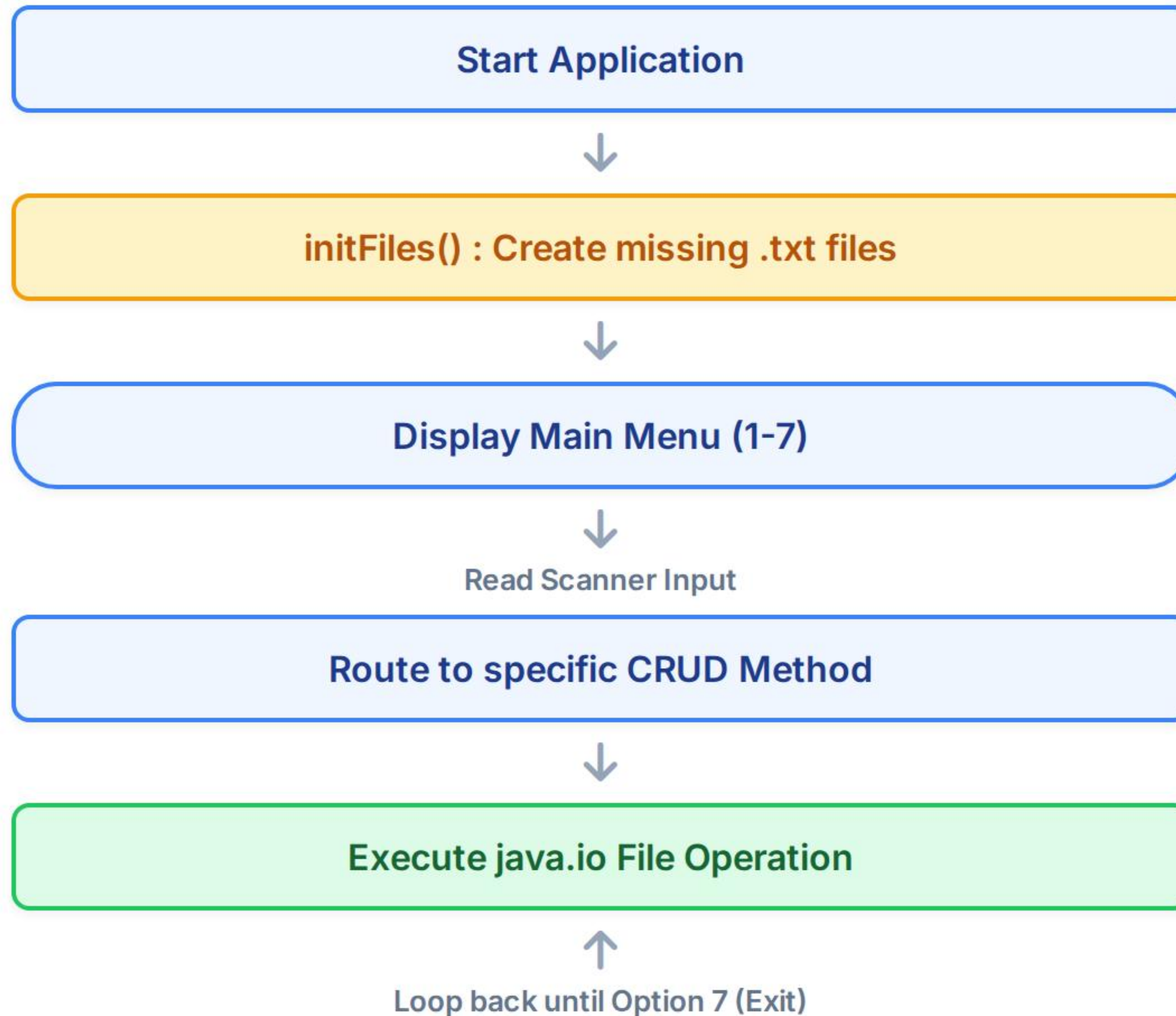
### Model & Storage Layer

The **Data Models** are represented by POJOs (Plain Old Java Objects): `Book.java` and `Member.java`, encapsulating object state.

The **Storage Layer** utilizes `java.io`. Data is serialized into a CSV format via custom `toFileString()` methods and written directly to local flat files.

# 7. Design Diagrams (Workflow)

## Application Workflow Diagram

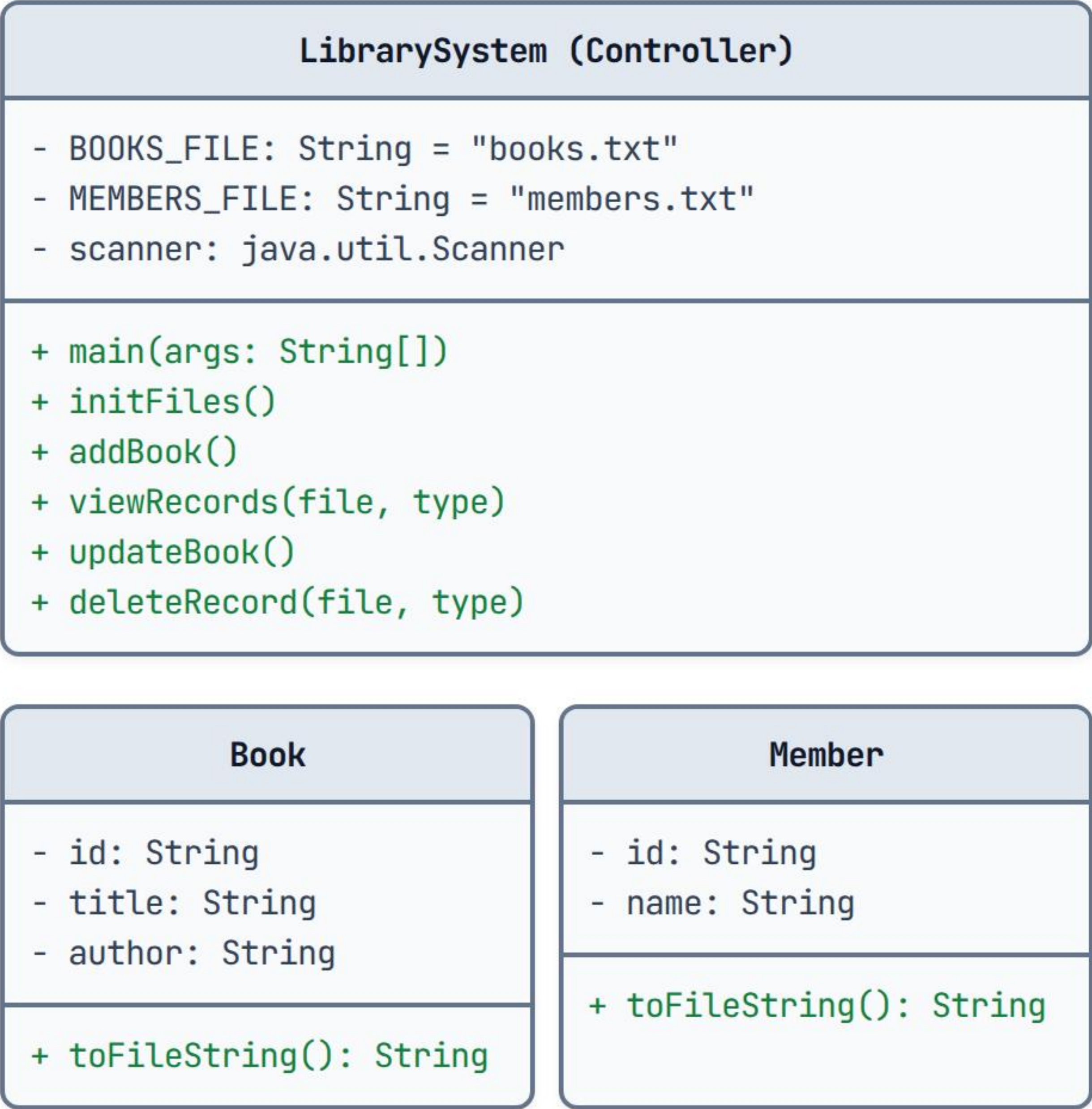


## System Use Cases

-  **Admin** adds new Book
-  **Admin** views Book Library
-  **Admin** updates Book Details
-  **Admin** deletes lost Book
-  **Admin** registers Member
-  **Admin** views Member List

# 7. Design Diagrams (Class & Storage)

## UML Class Diagram



## Flat-File Storage Schema

(Acts as ER representation in a database-less system)

| books.txt (Comma Separated) |                |                  |
|-----------------------------|----------------|------------------|
| ID (PK)                     | Title          | Author           |
| 101                         | Effective Java | Joshua Bloch     |
| 102                         | Clean Code     | Robert C. Martin |

| members.txt (Comma Separated) |             |
|-------------------------------|-------------|
| ID (PK)                       | Name        |
| M001                          | Alice Smith |
| M002                          | Bob Jones   |

# 8. Design Decisions & Rationale

---

## Why Java.io over JDBC/SQL?

The primary goal was absolute portability. Setting up MySQL requires local installation, drivers, and port configuration. **File I/O provides a "plug-and-play" experience** that works immediately upon compilation, ideal for academic and lightweight utility projects without administrative overhead.

## The Temp-File Swap Strategy

Standard text files do not support inline modification of specific lines. To update or delete a record, we implemented a highly reliable structural approach:

1. Read original file line-by-line.
2. Write modifications (or skip lines for deletion) to a `temp.txt` file.
3. Delete original file & rename `temp.txt` to original name.

# 9. Implementation Details

---

## Appending Data

Used `FileWriter(filename, true)` to enable append mode, wrapped in a `PrintWriter` to easily output complete strings via `pw.println(data)`.

## Reading & Parsing

Employed `BufferedReader` for efficient reading. Used `String.split(",")` to break down the CSV format into an array for UI formatting.

## Resource Management

Implemented Java 7's **Try-With-Resources** blocks extensively, guaranteeing streams close automatically to prevent file locking.

```
// Implementation: Temp-File Update Logic try
(BufferedReader br = new BufferedReader(new
FileReader(inputFile)); PrintWriter pw = new
PrintWriter(new FileWriter(tempFile))) { String
line; while ((line = br.readLine()) != null) {
String[] parts = line.split(","); if
(parts[0].equals(idToUpdate)) { // Write new
modified data to temp pw.println(newId + "," +
newTitle); } else { // Write original unmodified
data pw.println(line); } } } // Auto-closed streams
```

# 10. Execution Results (Console Output)

---

```
$ java LibrarySystem
```

```
--- Library System ---
```

```
1. Add Book
```

```
2. View All Books
```

```
3. Update Book ...
```

```
Choice: 1
```

```
Enter Book ID: 101
```

```
Enter Title: Effective Java
```

```
Enter Author: Joshua Bloch
```

```
Book saved!
```

```
--- Books List ---
```

```
ID: 101 | Effective Java | Joshua Bloch
```

# 11. Testing Approach

---

## Unit & Functional Testing

---

Each CRUD function was isolated and tested manually via the console:

- **Create:** Verified data appeared exactly as typed in the `.txt` file without trailing spaces.
- **Read:** Ensured the `split(",")` logic didn't throw `ArrayIndexOutOfBoundsException` exceptions on malformed lines.
- **Update:** Tested keeping old values by intentionally submitting blank string inputs during the update prompt.

## Edge Case Handling

---

Specific scenarios were tested to ensure robustness:

- **First Run:** Deleted all text files before running to ensure `initFiles()` caught the missing files and created them without crashing.
- **Invalid IDs:** Attempted to delete or update non-existent book IDs to verify the "ID not found" error messages triggered and state remained unchanged.

# 12. Challenges Faced

---

 **The Scanner Newline Bug:** Initially, skipping between numerical inputs (like `nextInt()`) and string inputs caused the Scanner to consume empty newline characters, inadvertently skipping user prompts.

**Resolution:** Standardized purely on `scanner.nextLine()` for all inputs to ensure buffer clearing.

 **OS File Locking:** Early builds crashed during the Update phase because the original `books.txt` file couldn't be deleted while a `FileReader` was theoretically still pointing to it in memory.

**Resolution:** Adopting Try-With-Resources ensured the reader was forcefully closed before the `file.delete()` command executed.

**Delimiter Conflicts:** If a user entered a Book Title containing a comma (e.g., "Java, The Complete Reference"), it broke the CSV split logic upon reading.

**Resolution:** Required establishing input rules to avoid commas in strings, keeping parsing simple.

## 13. Key Takeaways

---

### Mastery of Java I/O

Gained a deep, practical understanding of `FileWriter`, `BufferedReader`, and manipulating the local file system.

### Clean Architecture

Realized that separating the data model layer from the logic/controller layer makes the codebase drastically easier to maintain.

## 14. Future Scope

---

### Graphical User Interface

Transitioning from the command-line to a desktop GUI using Java Swing or JavaFX for a modern experience.

### Database Migration

Upgrading backend storage from text files to a relational database like SQLite using JDBC for true scalability.

# 15. References & Documentation

## **Oracle Java SE Documentation**

Official specifications for the `java.io` package, Stream classes, and Exception handling paradigms.

## **"Effective Java" by Joshua Bloch**

Consulted for best practices regarding Object-Oriented design patterns and resource management (specifically Try-With-Resources).

## **Stack Overflow Community**

Referenced for solutions regarding Scanner class newline behaviors and File renaming/deletion operations across different OS environments.